

Fundamentos de la Visión por Computador

Detección de tanques de almacenamiento de petróleo en imágenes satelitales:
métodos deterministas de OpenCV frente a una CNN (YOLOv8)

Gotzon Viteri Fuente

6 de agosto de 2026

Índice

1. Introducción y Objetivo General	2
1.1. Definición del problema	2
1.2. Estado del arte y trabajos relacionados	2
1.3. Valor añadido del proyecto	3
2. Metodología y Desarrollo	3
2.1. Conjunto de datos	3
2.2. Arquitectura del proyecto	4
2.3. Protocolo de evaluación común	4
2.4. Técnicas deterministas evaluadas	5
2.5. Modelo de referencia: YOLOv8	6
3. Resultados y Conclusiones	6
3.1. Tabla comparativa (20 imágenes de validación)	6
3.2. Análisis de los fallos del sistema determinista	6
3.3. Un fallo de diseño real, encontrado y corregido durante el desarrollo	7
3.4. Validación externa del resultado con YOLO	7
3.5. Ejemplo visual	7
3.6. Líneas de mejora futuras	8
Anexo: reproducibilidad	9

1. Introducción y Objetivo General

1.1. Definición del problema

El problema abordado es la **detección de tanques de almacenamiento de petróleo** en imágenes de satélite de alta resolución. Dada una imagen aérea de una instalación industrial o portuaria, el sistema debe localizar cada tanque presente mediante una caja delimitadora (*bounding box*), sin necesidad de clasificar subtipos de tanque ni estimar su volumen.

Los tanques de almacenamiento son estructuras cilíndricas que, vistas desde arriba, aparecen como **círculos o elipses casi perfectos**, de tamaño relativamente uniforme dentro de una misma instalación pero muy variable entre instalaciones distintas (de pocos metros a más de 100 m de diámetro). Se agrupan característicamente en “granjas de tanques” — filas y rejillas muy densas de decenas o cientos de unidades — lo que convierte la **separación de instancias contiguas** en el principal reto técnico del problema, por encima de la propia detección de la forma circular.

Este tipo de detección tiene aplicaciones reales en inteligencia geoespacial: estimación de reservas estratégicas de petróleo a partir de la sombra proyectada por el techo flotante de los tanques, monitorización de actividad industrial, o vigilancia de infraestructuras críticas — todas ellas tareas que hoy dependen de imagen satelital comercial y que motivaron a Airbus Defense and Space a publicar el dataset utilizado en este proyecto.

1.2. Estado del arte y trabajos relacionados

Se han identificado y consultado las siguientes referencias directamente relacionadas con este problema y, en varios casos, con este mismo dataset:

1. **Hough Circle Transform** [1] — la técnica clásica de referencia para detectar formas circulares mediante acumulación de votos en un espacio de parámetros (centro, radio), implementada en OpenCV como `cv2.HoughCircles`. Es el punto de partida natural para cualquier detector de objetos circulares, pero **no discrimina textura de fondo**: en escenas con muchos bordes (carreteras, parcelas agrícolas) genera un número inmanejable de falsos círculos si se ajusta para ser sensible a objetos pequeños.
2. **Faudi, J.** — “*Oil Storage Detection on Airbus Imagery with YOLOX*” [2], curador del propio dataset. Trocea las imágenes de 2560×2560 en *tiles* de 640×640 con solape de 64 px (protocolo que hemos replicado en este proyecto) y entrena YOLOX-s durante solo 10 épocas, obteniendo $AP@0.5 = 0.856$. *Limitación*: no compara contra ningún método clásico ni analiza en qué tipo de escena falla el modelo.
3. **Ouzounis, G.** — “*Oil-Storage Tank Instance Segmentation with Mask R-CNN*” [3]. Convierte las cajas del mismo dataset en máscaras circulares sintéticas y entrena Mask R-CNN sobre recortes de 128×128 . Reporta cualitativamente el mismo problema estructural que hemos encontrado nosotros: “leakage” (fuga) al separar tanques contiguos por falta de contexto en recortes pequeños. *Limitación*: no aporta métricas cuantitativas de detección, solo observaciones cualitativas.
4. **Wang et al.** — “*Storage tank detection in remote sensing images based on circular bounding boxes and large selective kernel*” [4], Scientific Reports (2025). Propone sustituir la caja rectangular estándar de un detector tipo YOLO por una parametrización circular explícita (centro + radio) y un kernel selectivo de campo receptivo amplio, alineando mejor la representación del modelo con la forma real del objeto. Es la línea de mejora más prometedora

que hemos identificado para superar el techo de $F1 \approx 0.84$ obtenido con una caja rectangular genérica, aunque no hemos podido confirmar si sus experimentos usan este mismo dataset (artículo con acceso restringido).

5. **Beucher, S. & Meyer, F.** — algoritmo de segmentación *watershed* controlado por marcadores [5] (técnica clásica de procesamiento morfológico, disponible en `cv2.watershed`), el método de referencia en visión por computador clásica para separar objetos convexos que se tocan (su aplicación más citada es la separación de células en microscopía). Es la base de uno de los experimentos deterministas de este proyecto.

Ningún trabajo consultado reporta un *baseline* puramente determinista (sin aprendizaje) para este dataset con el que comparar cuantitativamente. Esa ausencia es precisamente el hueco que este proyecto cubre.

1.3. Valor añadido del proyecto

Frente a los antecedentes anteriores, este proyecto no se limita a entrenar un detector más: aporta

- Un **barrido sistemático de 7 técnicas deterministas** de complejidad creciente (Hough circular, Hough tileado, contornos + circularidad, *blob detection*, segmentación *watershed* controlada por marcadores, detección multiescala Diferencia-de-Gaussianas) más un post-proceso de filtrado por densidad — todas evaluadas bajo el **mismo protocolo exacto** (mismo *split*, mismo umbral de IoU, mismo código de métricas) que el modelo de referencia basado en CNN, algo que ninguna de las referencias anteriores ofrece.
- Una **cuantificación del techo real del enfoque clásico** en este problema concreto ($F1 \approx 0.354$, frente a $F1 \approx 0.843$ de una CNN *fine-tuned*) y, más importante, un **diagnóstico de por qué** se produce ese techo: no es un problema de ajuste de hiperparámetros (se han probado más de 150 configuraciones distintas en total) sino una limitación estructural — los detectores de forma clásicos razonan objeto a objeto a partir de bordes o umbrales locales, y el 50 % de las imágenes del dataset contienen más de 100 tanques colocados en rejillas muy densas, el escenario exacto en el que separar instancias contiguas sin contexto de alto nivel es más difícil.
- La confirmación (Sección 3.4) de que nuestro resultado con YOLOv8 es coherente con el resultado publicado por el propio curador del dataset usando YOLOX y un protocolo de *tiling* casi idéntico, lo que valida la metodología de evaluación seguida.

2. Metodología y Desarrollo

2.1. Conjunto de datos

Se ha utilizado el dataset **Airbus Oil Storage Detection** [7] (Airbus DS Intelligence / Scale AI), publicado bajo licencia Creative Commons BY-NC-SA 4.0.

Limpieza de datos: se detectó que algunas anotaciones del CSV tienen $y_2 < y_1$ (altura negativa, p. ej. una fila con $h = -77$), por lo que el *parser* de anotaciones normaliza cada caja tomando ($\min(x_1, x_2), \min(y_1, y_2), \max(x_1, x_2), \max(y_1, y_2)$) en vez de asumir que las coordenadas ya vienen ordenadas.

Partición: *split* fijo del 80/20 a nivel de imagen completa (no de anotación individual, para evitar fuga de información entre *train* y *val*), con semilla aleatoria fija (42), generando 78 imágenes de

Propiedad	Valor
Nº de imágenes	98 (+ 5 adicionales sin anotar en extras/)
Resolución de imagen	2560 × 2560 px
Resolución de suelo	~1.2–1.5 m/píxel (satélite SPOT)
Condiciones de adquisición	Óptico, luz natural diurna, ubicaciones mundiales diversas (iluminación/sombra variable), imagen fija ortorectificada (sin movimiento)
Nº total de anotaciones	13 592 cajas, clase única oil-storage-tank
Tanques por imagen	mín. 14, mediana 99.5, máx. 893
Tamaño de tanque (lado de caja)	mín. 3 px, mediana 18 px, máx. 108 px

Tabla 1: Características del dataset Airbus Oil Storage Detection.

entrenamiento/ajuste y 20 de validación. El mismo *split* se reutiliza para **todos** los métodos comparados, de forma que los números de la Sección 3 son directamente comparables entre sí.

2.2. Arquitectura del proyecto

El proyecto se organiza en módulos reutilizables (**src/**) y *scripts* ejecutables (**scripts/**):

src/dataset.py carga/parseo de anotaciones, *split* train/val, lectura/escritura de imagen segura en Windows.

src/evaluate.py IoU, *matching greedy*, métricas P/R/F1, NMS genérico.

src/tiling.py partición de una imagen grande en *tiles* con solape.

src/hough_detector.py Hough Circles sobre imagen completa + *grid search*.

src/classic_methods.py 7 variantes deterministas adicionales, todas tileadas.

src/yolo_data.py conversión del dataset a formato YOLO (tiling + etiquetas).

src/yolo_detector.py entrenamiento e inferencia tileada de YOLOv8.

src/visualize.py superposición de cajas GT/predicción sobre la imagen.

Los *scripts* de **scripts/** orquestan el ajuste de hiperparámetros (*grid search*), la evaluación sobre validación y la generación de tablas y figuras comparativas para cada método.

Librerías utilizadas: OpenCV (**opencv-python**) para todo el procesamiento de imagen de bajo/medio nivel (filtrado, umbralización, transformada de Hough, **SimpleBlobDetector**, morfología, *watershed*), NumPy para álgebra vectorizada (IoU, NMS), pandas para el parseo de anotaciones, **scikit-image** [10] para detección de *blobs* multiescala y **peak_local_max/regionprops**, SciPy para el etiquetado de componentes conexas, **ultralytics** (YOLOv8 [8], sobre PyTorch con aceleración CUDA) como referencia de red neuronal, y matplotlib/OpenCV [9] para las visualizaciones.

2.3. Protocolo de evaluación común

Para que la comparación entre técnicas tan distintas sea honesta, todas comparten:

- **Emparejamiento *greedy* por IoU**: cada predicción se empareja, por orden de confianza (o de llegada si el detector no aporta confianza, como los métodos clásicos), con la anotación real no usada de mayor solapamiento, aceptando el emparejamiento solo si $\text{IoU} \geq 0,5$.
- **Métricas**: precisión, *recall*, F1 (métrica principal para elegir hiperparámetros), IoU medio de los aciertos (mide *calidad de localización* independientemente de cuántos aciertos haya) y tiempo de inferencia por imagen.
- **Ajuste de hiperparámetros exclusivamente sobre *train***: cada método se afina por *grid search* maximizando F1 sobre una muestra de entrenamiento (8–12 imágenes según el experimento) y la configuración ganadora se congela antes de evaluarse, una única vez, sobre las 20 imágenes de validación.
- **NMS genérico** para fusionar detecciones solapadas entre *tiles* contiguos.

2.4. Técnicas deterministas evaluadas

Todas operan por **teselado** (*tiling*) de la imagen en ventanas de 640×640 px con 64 px de solape (salvo el primer experimento, que se hace sobre la imagen completa como punto de partida), replicando la misma estrategia de partición usada para YOLO, de forma que la comparación no esté sesgada por trabajar a distinta escala espacial.

1. **Hough Circles, imagen completa** — `cv2.HoughCircles` sobre la imagen de 2560×2560 tras realce con CLAHE y filtro de mediana. *Grid search* sobre `dp`, `param1`, `param2`, `minDist`.
2. **Hough Circles, tileado** — la misma transformada, pero corrida ventana a ventana y fusionando resultados con NMS.
3. **Contornos + circularidad + *watershed* local** — umbral adaptativo, contornos filtrados por circularidad ($4\pi \text{Área}/\text{Perímetro}^2$), con separación por *watershed* solo cuando un contorno es sospechosamente grande (probable fusión de varios tanques).
4. `cv2.SimpleBlobDetector` — detector de *blobs* nativo de OpenCV, configurado para buscar regiones estables a través de múltiples umbrales de brillo, filtrando por área, circularidad, convexidad y polaridad de color (*blobs* claros vs. oscuros).
5. **Segmentación *watershed* universal controlada por marcadores** — umbral global por percentil de brillo, *distance transform*, máximos locales como semillas de primer plano, fondo seguro por dilatación, y `cv2.watershed` sobre toda la máscara (no solo sobre *blobs* sospechosos, a diferencia de la técnica 3).
6. **Detección multiescala Diferencia-de-Gaussianas (DoG)** — `skimage.feature.blob_dog` [6], que barre un rango continuo de escalas (σ) en vez de trabajar a escala fija.
7. **Post-proceso: filtro por densidad de vecinos** — aplicado sobre el mejor detector (técnica 4), descarta detecciones sin ningún otro candidato cercano, bajo la hipótesis de que los tanques reales aparecen casi siempre en grupo.

El detalle experimento a experimento — incluyendo un bug real de diseño detectado y corregido durante el desarrollo (Sección 3.3) — está documentado íntegramente en el fichero `EXPERIMENTS.md` que acompaña al código fuente.

2.5. Modelo de referencia: YOLOv8

Como referencia de lo que consigue un enfoque de aprendizaje profundo con el mismo volumen de datos, se entrenó **YOLOv8n** (variante más pequeña de Ultralytics, preentrenada en COCO) mediante *fine-tuning*:

- Las imágenes de entrenamiento se trocean en *tiles* de 640×640 px con 64 px de solape (mismo esquema que los métodos clásicos), conservando solo los *tiles* con al menos un tanque, y descartando cajas con menos del 40 % de su área dentro del *tile*.
- Entrenamiento: 60 épocas, tamaño de imagen 640, sobre GPU NVIDIA RTX 3060 (CUDA).
- Inferencia sobre las imágenes de validación: se aplica el mismo teselado, se corre el modelo *tile* a *tile* y se fusionan las detecciones con NMS, re proyectando a las coordenadas de la imagen original de 2560×2560 — el mismo procedimiento, con el mismo código de teselado, que usan los métodos clásicos.

3. Resultados y Conclusiones

3.1. Tabla comparativa (20 imágenes de validación)

Método	Precisión	Recall	F1	IoU medio	Tiempo/img
Hough Circles, imagen completa	0.113	0.044	0.063	0.814	0.073 s
DoG multiescala	0.031	0.052	0.039	0.565	3.363 s
Contornos + circularidad + <i>watershed</i> local	0.144	0.107	0.123	0.743	0.193 s
Hough Circles, tileado	0.443	0.104	0.169	0.769	0.175 s
<i>Watershed</i> universal	0.287	0.217	0.247	0.657	0.794 s
<i>Blob detector</i> (grid inicial)	0.323	0.291	0.306	0.671	0.439 s
<i>Blob detector</i> (grid refinado)	0.584	0.254	0.354	0.672	0.429 s
<i>Blob detector</i> + filtro de densidad	0.626	0.248	0.355	0.671	0.428 s
YOLOv8n <i>fine-tuned</i>	0.812	0.876	0.843	0.809	0.373 s

Tabla 2: Comparativa de todos los métodos evaluados bajo el mismo protocolo, sobre las 20 imágenes de validación.

El mejor método determinista (*blob detector* afinado, con o sin filtro de densidad) multiplica por $\sim 5.6 \times$ el F1 del primer intento con Hough, pero se queda a **menos de la mitad** del F1 conseguido con YOLOv8.

3.2. Análisis de los fallos del sistema determinista

El hallazgo transversal a los ocho experimentos es que **el cuello de botella no es de precisión geométrica sino de *recall*/detección**: el IoU medio de los aciertos es comparable entre casi todos los métodos (0.56–0.81) e incluso similar al de YOLO (0.81). Es decir, cuando un método clásico sí encuentra un tanque, dibuja una caja tan buena como la de la red neuronal. El problema es que **no encuentra la mayoría de ellos**: incluso el mejor método clásico solo recupera el 25 % de los tanques reales, frente al 88 % de YOLO.

La causa raíz identificada es estructural: los detectores de forma clásicos (Hough, contornos, *blobs*, *watershed*) razonan **un objeto a la vez** a partir de bordes o umbrales locales. Este dataset tiene una mediana de ~ 100 tanques por imagen colocados en rejillas muy juntas (hasta

893 en la imagen más densa), y separar instancias que se tocan sin contexto de más alto nivel — el tipo de contexto que una CNN aprende directamente de ejemplos anotados — es precisamente el escenario donde estas técnicas fallan.

Un segundo patrón, más práctico, es que **la elección de técnica importa más que su ajuste fino**: pasar de Hough a *blob detection* (mismo *tiling*, mismo protocolo) duplicó el F1 sin cambiar nada más, simplemente porque el `SimpleBlobDetector` incorpora de forma nativa un filtro de **polaridad de brillo** (*blobs* claros sobre fondo oscuro) que Hough y DoG no tienen. Los tanques son, ante todo, los objetos más claros y compactos de la escena; cualquier técnica que ignore esa señal no supera $F1 \approx 0.17$.

3.3. Un fallo de diseño real, encontrado y corregido durante el desarrollo

Durante el experimento de *watershed* universal, la primera implementación devolvía sistemáticamente $F1 = 0.000$. El diagnóstico reveló que **nunca se marcaba explícitamente el fondo** antes de llamar a `cv2.watershed`: se dejaban esos píxeles con valor 0 (“desconocido”) en vez de asignarles una etiqueta de fondo seguro. Sin esa semilla, el algoritmo no tiene desde dónde crecer la región de fondo, y el 99.97 % de las regiones resultantes quedaban reducidas a fragmentos de 1 píxel. La corrección aplicó la receta estándar de *watershed* controlado por marcadores (fondo seguro por dilatación + frente seguro por los máximos locales + región desconocida entre ambos), tras lo cual el método pasó a producir resultados razonables ($F1 = 0.247$). Se documenta este episodio porque ilustra un error conceptual no trivial — y su corrección — más que un simple problema de hiperparámetros.

De forma similar, el experimento de DoG resultó ser el peor de todos ($F1 = 0.039$, incluso por debajo del Hough más ingenuo) precisamente por *no* incorporar el filtro de polaridad de brillo que sí tiene el *blob detector* — confirmando *post-hoc* la importancia de esa señal.

Por último, el filtro de densidad de vecinos, aplicado como post-proceso sobre el mejor método, no aportó mejora neta ($F1\ 0.354 \rightarrow 0.355$): aunque los tanques reales sí aparecen en grupo, **los falsos positivos también** (otras estructuras humanas agrupadas, como naves industriales), así que la densidad espacial por sí sola no distingue “cluster de tanques” de “cluster de tejados similares”.

3.4. Validación externa del resultado con YOLO

El resultado obtenido con YOLOv8n ($F1 = 0.843$) es consistente con el publicado por el propio curador del dataset [2], que con YOLOX-s y un esquema de *tiling* prácticamente idéntico (640×640 , solape de 64 px) obtiene $AP@0.5 = 0.856$ tras solo 10 épocas sin ajuste de hiperparámetros. La cercanía de ambos números, obtenidos de forma completamente independiente, respalda la validez del protocolo de evaluación seguido en este proyecto.

3.5. Ejemplo visual

La Figura 1 muestra, sobre una misma imagen de validación, las anotaciones reales (verde) frente a las predicciones de Hough tileado (azul) y de YOLOv8 (rojo): YOLO cubre casi la totalidad de los tanques reales, mientras que Hough solo detecta los tanques grandes y aislados, ignorando los clústeres densos.



Figura 1: Comparativa cualitativa sobre una imagen de validación: anotación real (verde), Hough tileado (azul), YOLOv8 (rojo).

3.6. Líneas de mejora futuras

- **Señal de color real** en vez de solo brillo en escala de grises: los métodos deterministas actuales solo distinguen “claro vs. oscuro”; incorporar el matiz/saturación (espacio HSV) podría separar tejados de tanque de otras superficies brillantes (hormigón, tejados residenciales) que hoy generan falsos positivos.
- **Cajas circulares explícitas** en el detector de red neuronal, siguiendo la propuesta de Wang et al. [4], en vez de la caja rectangular genérica de YOLO — debería mejorar la precisión de localización en tanques muy solapados.
- **Ensemble de métodos deterministas** (unión con NMS de *blob detector* + *watershed* universal): al usar lógicas de segmentación distintas sobre la misma señal de brillo, podrían tener falsos positivos no correlacionados y aciertos parcialmente complementarios.
- **Ajuste fino del lado YOLO**, que quedó deliberadamente fuera de alcance de este proyecto (se usó solo como referencia, no como objeto de optimización): barrido de umbral de confianza/NMS en inferencia, modelos más grandes (YOLOv8s/m), más épocas con aumentado de datos, o *tiles* más pequeños específicamente para mejorar el *recall* en los clústeres más densos.
- **Ampliación del dataset de entrenamiento**: con solo 78 imágenes de entrenamiento, un esquema de aprendizaje activo o la incorporación de más imágenes anotadas de la colección

completa de Airbus (este es solo un subconjunto de demostración) sería la palanca más directa para seguir mejorando el modelo de referencia.

Anexo: reproducibilidad

Todo el código fuente, comentado y organizado en módulos (`src/`) y *scripts* ejecutables (`scripts/`), se entrega junto a esta memoria. La bitácora experimental completa — cada configuración probada, su resultado y el razonamiento detrás de cada decisión — está en el fichero `EXPERIMENTS.md`. Las métricas crudas de cada experimento están en `results/*.json` y las visualizaciones comparativas (anotación real vs. predicción) en `results/figures/`.

Referencias

- [1] Duda, R. O. & Hart, P. E. (1972). *Use of the Hough Transformation to Detect Lines and Curves in Pictures*. Communications of the ACM. (Fundamento teórico de `cv2.HoughCircles`).
- [2] Faudi, J. — *Oil Storage Detection on Airbus Imagery with YOLOX*. Medium / Artificialis. <https://medium.com/artificialis/oil-storage-detection-on-airbus-imagery-with-yolox-9e38eb6f7e62>
Notebook asociado: <https://www.kaggle.com/code/jeffaudi/oil-storage-detection-on-airbus-imagery-with-yolox>
- [3] Ouzounis, G. — *Oil-Storage Tank Instance Segmentation with Mask R-CNN*. Medium. <https://medium.com/@georgios.ouzounis/oil-storage-tank-instance-segmentation-with-mask-r-cnn-77c94433045f>
- [4] Wang et al. (2025). *Storage tank detection in remote sensing images based on circular bounding boxes and large selective kernel*. Scientific Reports. <https://www.nature.com/articles/s41598-025-27919-5>
- [5] Beucher, S. & Meyer, F. (1993). *The morphological approach to segmentation: the watershed transformation*. En *Mathematical Morphology in Image Processing*, Marcel Dekker. (Fundamento teórico de `cv2.watershed`).
- [6] Lindeberg, T. (1998). *Feature Detection with Automatic Scale Selection*. International Journal of Computer Vision. (Fundamento teórico de la detección multiescala Laplaciano-de-Gaussiana/ Diferencia-de-Gaussianas implementada en `skimage.feature.blob_dog`).
- [7] Airbus DS Intelligence / Scale AI — *Airbus Oil Storage Detection Dataset*. Kaggle. <https://www.kaggle.com/datasets/airbusgeo/airbus-oil-storage-detection-dataset> (Licencia CC BY-NC-SA 4.0).
- [8] Jocher, G., Chaurasia, A. & Qiu, J. (2023). *Ultralytics YOLOv8* [software]. <https://github.com/ultralytics/ultralytics>
- [9] Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools. (`cv2.SimpleBlobDetector`, transformadas y operaciones morfológicas utilizadas en todo el proyecto).
- [10] Van der Walt, S. et al. (2014). *scikit-image: image processing in Python*. PeerJ. (`skimage.feature.peak_local_max`, `skimage.measure.regionprops`, `skimage.feature.blob_dog`).